Semana 11 · prática reduzida

ETL/ELT + dbt em 60 minutos

Você vai observar dados ruins falhando em testes, corrigir a camada staging, criar um mart e provar o resultado com evidências.

Entrega

PDF com respostas + prints. Faça também backup em JSON.

Regra

Não basta mostrar código: registre **FAIL → correção → PASS → resultado final**.

Não use dbt init. O projeto já vem configurado para economizar tempo.

Nome

yan ismael clemente alves

Turma

3° tec IA - tarde

Roteiro para caber em uma aula prática

0–10 min	instalar/validar ambiente + preparar DuckDB
10–20	rodar staging inicial e capturar a falha dos testes
20–35	corrigir TODO 1 e obter PASS
35–47	completar TODO 2 (mart)
47–55	dbt build + verificar resultado
55–60	evidências, conclusão, JSON e PDF

Se a instalação demorar, não pule etapas de qualidade: use o tempo restante e registre o ponto alcançado.

Abra o ZIP no VS Code e valide o ambiente

1 Extraia o ZIP e abra a pasta `aluno_projeto` no VS Code.

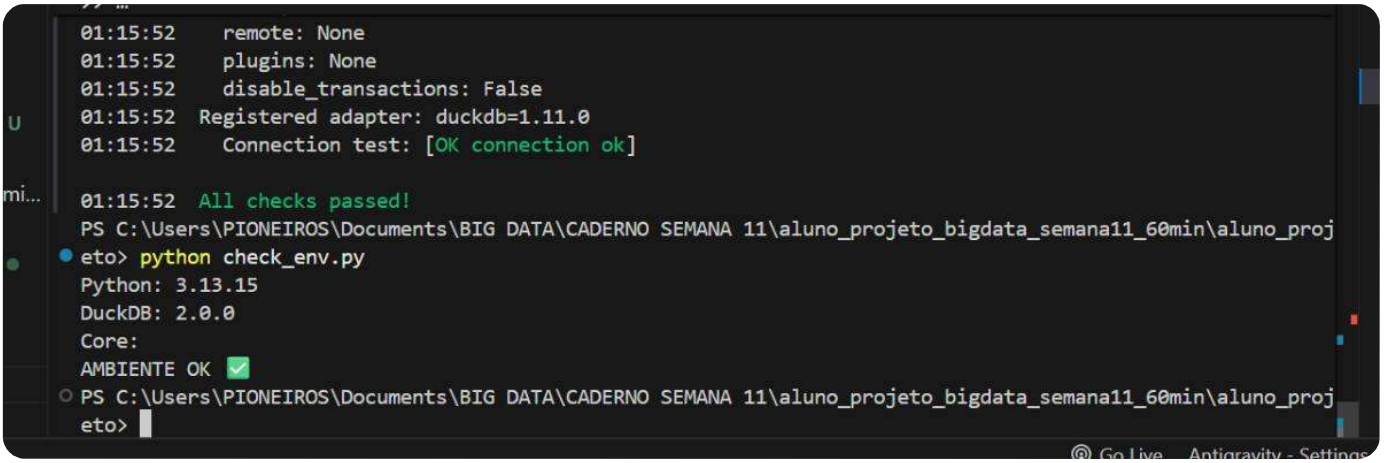
2 Abra **Terminal** → **Novo Terminal**. Confirme que o terminal está na pasta que contém `dbt_project.yml`.

```
python -m pip install -r requirements.txt
python check_env.py
python preparar_banco.py
dbt debug --profiles-dir .
```

Você deve ver **AMBIENTE OK**, **BANCO PREPARADO** e, no dbt debug, a conexão concluída com sucesso.

Evidência 1 · ambiente

Adicione um print do terminal com AMBIENTE OK / BANCO PREPARADO / conexão dbt.



Primeiro veja o problema antes de corrigir

O `stg_orders.sql` inicial ainda deixa `status` despadronizado e duas linhas sem `order_id`. Rode:

```
dbt run --select stg_orders stg_customers stg_products --profiles-dir .
dbt test --select stg_orders --profiles-dir .
```

Esperado: pelo menos os testes de `not_null` e/ou `accepted_values` devem falhar. Isso é intencional.

Quais testes falharam? Copie os nomes principais.

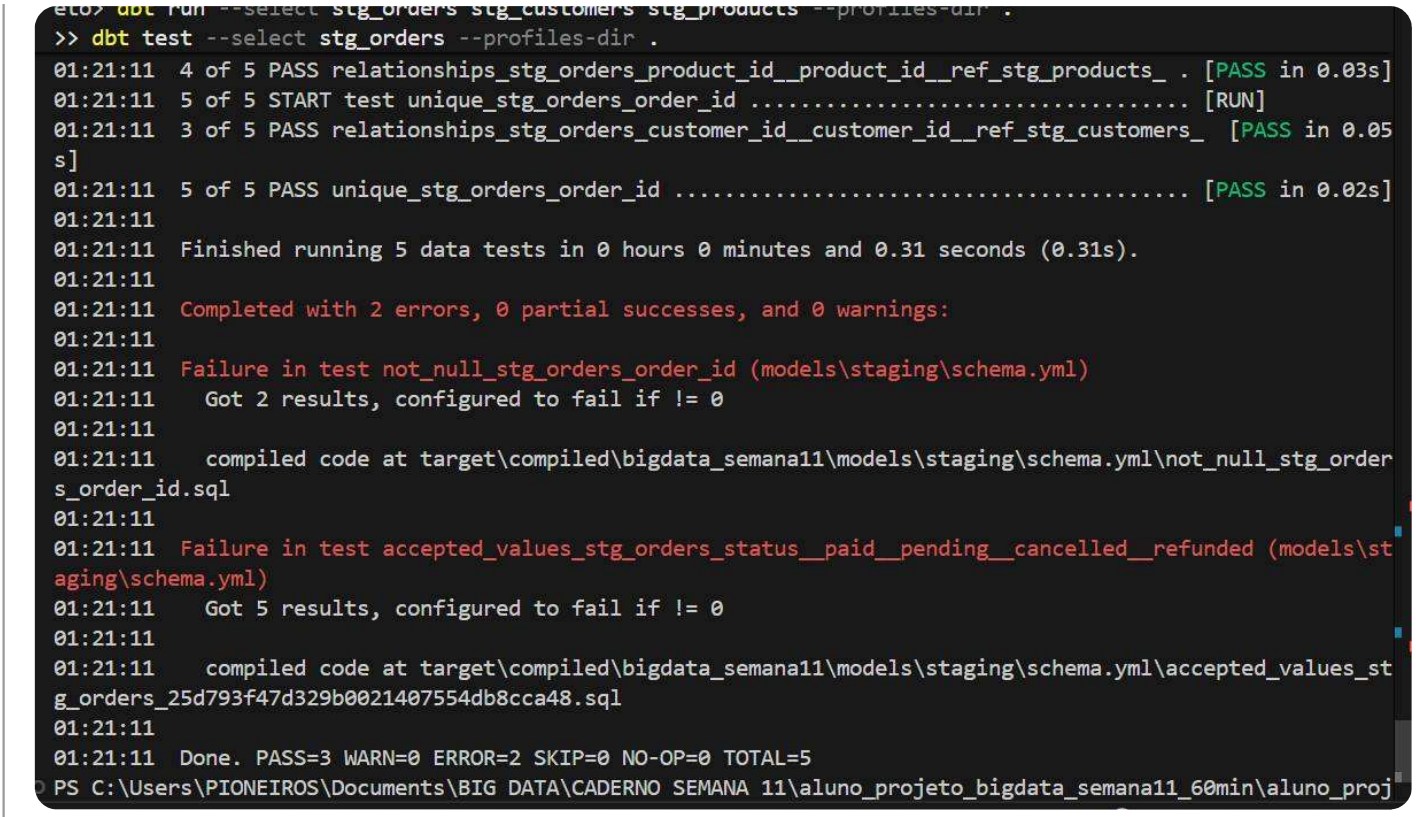
```
not_null_stg_orders_order_id e
accepted_values_stg_orders_status__paid__pending__cancelled__refunded
```

Por que o teste falhou?

```
O modelo stg_orders inicial apenas copia a tabela bruta sem tratamento. O teste falhou porque
existem 2 registros com order_id nulo e valores da coluna status com espaços extras e letras
maiúsculas (como ' Paid ' e 'PAID'), violando as regras definidas no schema.yml.
```

Evidência 2 · FAIL inicial

Print do terminal mostrando os testes com falha.


etc> dbt run --select stg_orders stg_customers stg_products --profiles-dir .
>> dbt test --select stg_orders --profiles-dir .
01:21:11 4 of 5 PASS relationships_stg_orders_product_id__product_id__ref_stg_products_ . [PASS in 0.03s]
01:21:11 5 of 5 START test unique_stg_orders_order_id ..... [RUN]
01:21:11 3 of 5 PASS relationships_stg_orders_customer_id__customer_id__ref_stg_customers_ [PASS in 0.05s]
01:21:11 5 of 5 PASS unique_stg_orders_order_id ..... [PASS in 0.02s]
01:21:11
01:21:11 Finished running 5 data tests in 0 hours 0 minutes and 0.31 seconds (0.31s).
01:21:11
01:21:11 Completed with 2 errors, 0 partial successes, and 0 warnings:
01:21:11
01:21:11 Failure in test not_null_stg_orders_order_id (models\staging\schema.yml)
01:21:11 Got 2 results, configured to fail if != 0
01:21:11
01:21:11 compiled code at target\compiled\bigdata_semana11\models\staging\schema.yml\not_null_stg_orders_order_id.sql
01:21:11
01:21:11 Failure in test accepted_values_stg_orders_status__paid__pending__cancelled__refunded (models\staging\schema.yml)
01:21:11 Got 5 results, configured to fail if != 0
01:21:11
01:21:11 compiled code at target\compiled\bigdata_semana11\models\staging\schema.yml\accepted_values_stg_orders_25d793f47d329b0021407554db8cca48.sql
01:21:11
01:21:11 Done. PASS=3 WARN=0 ERROR=2 SKIP=0 NO-OP=0 TOTAL=5
PS C:\Users\PIONEIROS\Documents\BIG DATA\CADERNO SEMANA 11\aluno_projeto_bigdata_semana11_60min\aluno_proj

Corrija o staging: padronizar antes de entregar ao restante do projeto

Abra `models/staging/stg_orders.sql` e substitua o SELECT atual por:

```
select
    cast(order_id as integer) as order_id,
    cast(customer_id as integer) as customer_id,
    cast(product_id as integer) as product_id,
    cast(total as decimal(12,2)) as total,
    cast(order_date as date) as order_date,
    lower(trim(status)) as status
from {{ source('raw', 'orders') }}
where order_id is not null
```

O que cada mudança faz?

- `cast`: garante tipos previsíveis.
- `trim`: remove espaços extras.
- `lower`: padroniza maiúsculas/minúsculas.
- `where order_id is not null`: remove registro sem chave.

```
dbt run --select stg_orders --profiles-dir .
dbt test --select stg_orders --profiles-dir .
```

Esperado agora: PASS.

Explique em 2–4 linhas por que staging não é só "copiar a tabela bruta".

```
A camada staging não é apenas uma cópia direta porque é nela que garantimos a qualidade e
consistência dos dados: definimos tipos previsíveis (cast), padronizamos textos removendo
espaços e diferenças de maiúsculas/minúsculas (lower e trim) e descartamos registros
corrompidos ou nulos. Isso garante que as camadas de negócio recebam apenas dados confiáveis.
```

Evidência 3 · PASS após correção

Print do terminal com os testes passando.

```
>> dbt test --select stg_orders --profiles-dir .
02:04:48  Running with dbt=1.11.15
02:04:48  Registered adapter: duckdb=1.11.0
02:04:49  Found 4 models, 13 data tests, 3 sources, 500 macros
02:04:49
02:04:49  Concurrency: 2 threads (target='dev')
02:04:49
02:04:49  1 of 5 START test accepted_values_stg_orders_status__paid__pending__cancelled__refunded  [RUN]
02:04:49  2 of 5 START test not_null_stg_orders_order_id ..... [RUN]
02:04:49  2 of 5 PASS not_null_stg_orders_order_id ..... [PASS in 0.07s]
02:04:49  1 of 5 PASS accepted_values_stg_orders_status__paid__pending__cancelled__refunded  [PASS in 0.08
s]
02:04:49  3 of 5 START test relationships_stg_orders_customer_id__customer_id__ref_stg_customers_  [RUN]
02:04:49  4 of 5 START test relationships_stg_orders_product_id__product_id__ref_stg_products_  [RUN]
02:04:49  4 of 5 PASS relationships_stg_orders_product_id__product_id__ref_stg_products_ . [PASS in 0.04s]
02:04:49  3 of 5 PASS relationships_stg_orders_customer_id__customer_id__ref_stg_customers_  [PASS in 0.05
s]
02:04:49  5 of 5 START test unique_stg_orders_order_id ..... [RUN]
02:04:49  5 of 5 PASS unique_stg_orders_order_id ..... [PASS in 0.02s]
02:04:49
02:04:49  Finished running 5 data tests in 0 hours 0 minutes and 0.31 seconds (0.31s).
02:04:49
02:04:49  Completed successfully
02:04:49
02:04:49  Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=5
PS C:\Users\PIONEIROS\Documents\BIG DATA\CADERNO SEMANA 11\aluno_projeto_bigdata_semana11_60min\aluno_proj
```

Crie o modelo final de receita diária

Abra `models/marts/fct_revenue_daily.sql` e substitua pelo código:

```
select
    order_date,
    count(order_id) as total_pedidos,
    round(sum(total), 2) as receita_total,
    round(avg(total), 2) as ticket_medio
from {{ ref('stg_orders') }}
where status = 'paid'
group by order_date
```

Observe o ref(): ele cria a dependência `stg_orders → fct_revenue_daily`. O dbt passa a conhecer a ordem correta do DAG.

Qual a diferença entre `source('raw','orders')` e `ref('stg_orders')`?

```
O source('raw','orders') referencia uma tabela bruta externa carregada diretamente no banco de dados, sem garantias de limpeza ou rastreabilidade. Já o ref('stg_orders') faz referência a um modelo interno já tratado pelo dbt, permitindo que a ferramenta construa o gráfico de linhagem (DAG) e garanta que o staging seja executado antes do mart.
```

```
dbt run --select fct_revenue_daily --profiles-dir .
```

Execute modelos + testes na ordem do DAG

```
dbt build --profiles-dir .
python verificar_resultado.py
```

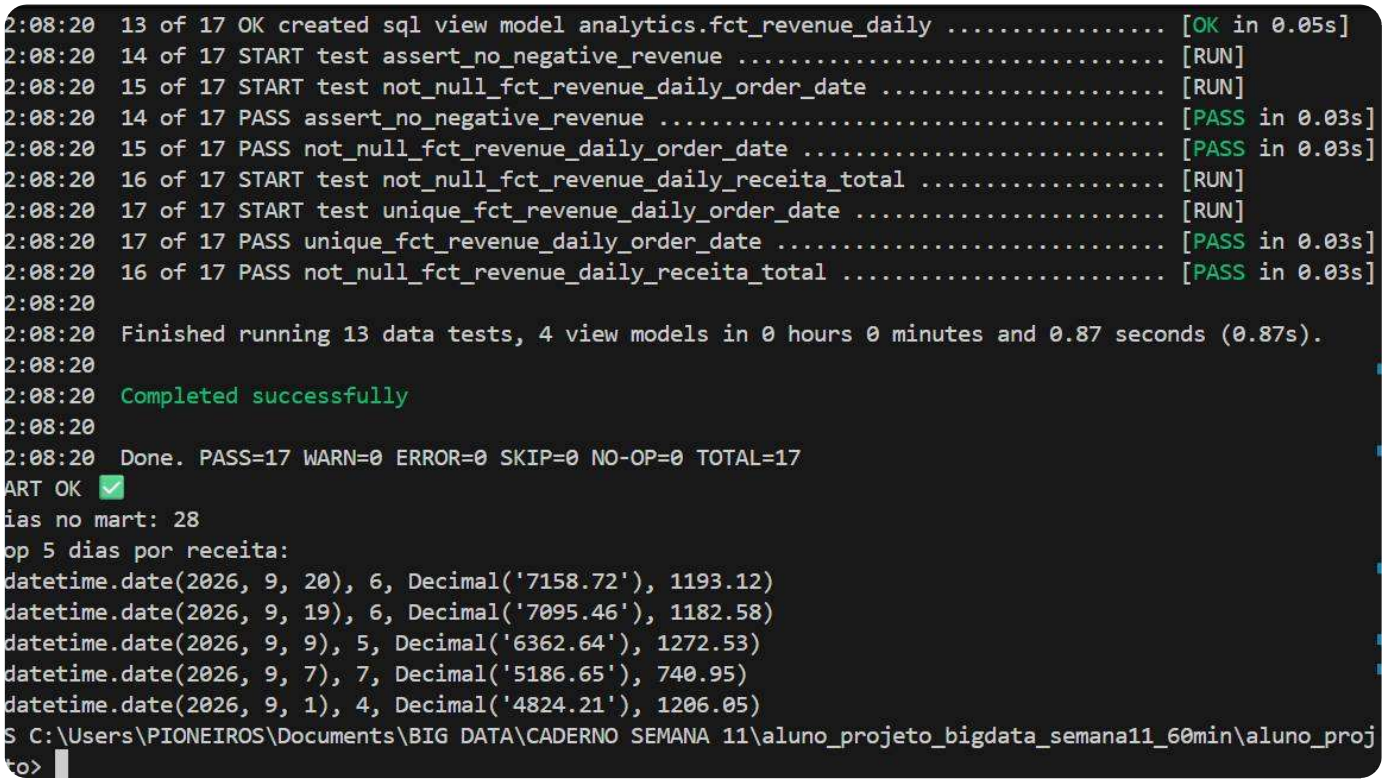
Resultado esperado: build concluído sem falhas e `MART OK` com os 5 dias de maior receita.

O que o `dbt build` fez além de executar os SELECTs?

Além de compilar e executar os SELECTs para criar as views de staging e marts, o dbt build orquestrou a execução conjunta de todos os 13 testes de qualidade de dados (como chaves únicas, não nulos, integridade referencial e valores aceitos), respeitando a ordem de dependências do grafo (DAG) para garantir que modelos só fossem criados se os dados estivessem válidos.

Evidência 4 · dbt build

Print mostrando o resumo do build sem falhas.



Evidência 5 · mart final

Print do `verificar_resultado.py` com MART OK e linhas de receita.

```
2:08:20  13 of 17 OK created sql view model analytics.fct_revenue_daily ..... [OK in 0.05s]
2:08:20  14 of 17 START test assert_no_negative_revenue ..... [RUN]
2:08:20  15 of 17 START test not_null_fct_revenue_daily_order_date ..... [RUN]
2:08:20  14 of 17 PASS assert_no_negative_revenue ..... [PASS in 0.03s]
2:08:20  15 of 17 PASS not_null_fct_revenue_daily_order_date ..... [PASS in 0.03s]
2:08:20  16 of 17 START test not_null_fct_revenue_daily_receita_total ..... [RUN]
2:08:20  17 of 17 START test unique_fct_revenue_daily_order_date ..... [RUN]
2:08:20  17 of 17 PASS unique_fct_revenue_daily_order_date ..... [PASS in 0.03s]
2:08:20  16 of 17 PASS not_null_fct_revenue_daily_receita_total ..... [PASS in 0.03s]
2:08:20
2:08:20  Finished running 13 data tests, 4 view models in 0 hours 0 minutes and 0.87 seconds (0.87s).
2:08:20
2:08:20  Completed successfully
2:08:20
2:08:20  Done. PASS=17 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=17
ART OK ✅
ias no mart: 28
op 5 dias por receita:
datetime.date(2026, 9, 20), 6, Decimal('7158.72'), 1193.12)
datetime.date(2026, 9, 19), 6, Decimal('7095.46'), 1182.58)
datetime.date(2026, 9, 9), 5, Decimal('6362.64'), 1272.53)
datetime.date(2026, 9, 7), 7, Decimal('5186.65'), 740.95)
datetime.date(2026, 9, 1), 4, Decimal('4824.21'), 1206.05)
S C:\Users\PIONEIROS\Documents\BIG DATA\CADERNO SEMANA 11\aluno_projeto_bigdata_semana11_60min\aluno_proj
to>
```

Fechamento e entrega

Em até 6 linhas: o que esta prática demonstrou sobre ELT + dbt?

A prática demonstrou o poder do paradigma ELT com dbt: os dados brutos foram carregados diretamente no banco DuckDB e transformados via SQL dentro da própria engine analítica. O dbt permitiu separar claramente a responsabilidade de limpeza no Staging da agregação de negócio nos Marts. Além disso, os testes automatizados garantiram a confiabilidade do pipeline (observando falhas antes e aprovações após as correções) e o grafo de dependências

- [x] Tenho print do ambiente.
- [x] Tenho print do FAIL inicial.
- [x] Tenho print do PASS após staging.
- [x] Tenho print do dbt build.
- [x] Tenho print do mart final.

Antes de fechar: clique em **Baixar JSON** para backup. Depois use **Imprimir / Salvar PDF**. Confira se os cinco prints aparecem no PDF antes de enviar.

Troubleshooting rápido

Problema	O que tentar
`dbt` não reconhecido	`python -m pip install -r requirements.txt` e feche/reabra o terminal.
Profile não encontrado	Use sempre `--profiles-dir .` e confirme que está na pasta do projeto.
raw.orders não existe	Rode `python preparar_banco.py`.
Teste ainda falha após TODO 1	Confirme `lower(trim(status))` e `where order_id is not null`.
Mart não tem receita_total	Você ainda está com o TODO 2 antigo; substitua todo o conteúdo do arquivo.
PDF sem print	Adicione a imagem no caderno e aguarde a prévia aparecer antes de imprimir.

Se houve erro, registre a mensagem e como resolveu.